SRM Souss-Massa

Gestion de flotte et de logistique

Guide de deploiement

Backoffice SRM Fleet sur serveur interne

Document technique interne

Redige par

Oussama BENAZZOUZ • **TAKI Fadlouallah**

27 août 2026

Table des matières

1 Objectif et principes 1
1.1 Chaine des composants 1
2 Prerequis du serveur 2
3 Recuperation du code 2
4 Configuration des secrets (.env) 2
5 Restreindre l'acces au reseau interne 2
5.1 Defense en profondeur (recommande) 3
6 Lancement 3
7 Premiere connexion 3
8 Persistance et sauvegardes 3
9 Mises a jour 4
10 Supervision 4
11 Checklist de securite (remise du projet) 4

1 Objectif et principes

Ce guide decrit le deploiement du backoffice **SRM Fleet** sur un serveur interne de l'entreprise. La plateforme est concue pour un usage **strictement interne** : elle n'est jamais exposee sur Internet.

- Toute la plateforme tourne en conteneurs (Docker Compose) : base de donnees, backend et frontend.
- **Seul le frontend** est accessible aux utilisateurs, et **uniquement sur le reseau interne**.
- Le **backend** et la **base de donnees** ne sont pas publies sur l'hote. Ils ne sont joignables que par le reseau interne de Docker. Le frontend (Nginx) relaie les appels `/api/` vers `backend:8080`.
- Aucun secret n'est stocke dans le depot. Tout passe par le fichier `.env` du serveur.

1.1 Chaine des composants

> Poste utilisateur (LAN) → **Frontend Nginx** (port 80) → relais `/api/` → **Backend Spring Boot** (8080) → **PostgreSQL** (5432).
> Les fleches de droite restent internes au reseau Docker : elles ne traversent jamais l'hote.

2 Prerequis du serveur

- Linux avec **Docker Engine** et **Docker Compose v2**.
- Serveur raccorde au reseau interne (adresse LAN, par exemple `10.0.0.10`), sans aucune redirection de port depuis le pare-feu peripherique et sans adresse IP publique routable.
- Environ 2 vCPU et 2 a 4 Go de RAM pour une charge moderee. Prevoir de l'espace disque pour la base de donnees et les pieces jointes.

3 Recuperation du code

```
git clone <URL_DU_DEPOT_INTERNE> srm-fleet
cd srm-fleet
git checkout main      # ou le tag de version a deployer
```

4 Configuration des secrets (`.env`)

```
cp .env.example .env
```

Renseigner des valeurs fortes et propres a la production :

Variable	Recommandation
POSTGRES_PASSWORD	Mot de passe aleatoire long.
JWT_SECRET	Au moins 32 octets aleatoires, par exemple `openssl rand -base64 48`. Le backend refuse de demarrer si la cle est trop courte.
APP_ADMIN_INITIAL_PASSWORD	Mot de passe fort pour le compte `admin`, a changer apres la premiere connexion.
JWT_EXPIRATION_MS	Duree de validite des jetons, par exemple `86400000` (24 heures).

> Le fichier `.env` ne doit jamais etre versionne (il est deja dans `.gitignore`). Restreindre ses droits sur le serveur : `chmod 600 .env`.

5 Restreindre l'acces au reseau interne

Le fichier `docker-compose.yml` par defaut publie les ports sur toutes les interfaces (`0.0.0.0`), ce qui convient au developpement mais pas a la production. On ajoute un fichier de surcharge de production qui expose le frontend uniquement sur l'IP interne du serveur et ne publie ni le backend ni la base.

Creer `docker-compose.prod.yml` :

```
services:
  db:
    ports: !reset []          # la base reste interne au reseau Docker
  backend:
    ports: !reset []          # le backend n'est joignable que via le frontend
  frontend:
```

```
    ports: !reset
      - "10.0.0.10:80:80"     # remplacer par l'IP interne du serveur
```

5.1 Defense en profondeur (recommande)

- Pare-feu de l'hote : n'autoriser le port 80 que depuis le sous-reseau interne.

```
ufw default deny incoming
ufw allow from 10.0.0.0/8 to any port 80 proto tcp
ufw enable
```

- Optionnel : placer le service derriere le reverse proxy interne de l'entreprise avec TLS (certificat de l'autorite interne), en n'acceptant que le trafic du LAN.

6 Lancement

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Verifications :

```
docker compose ps                 # tous les services "healthy"
curl -f http://localhost:80/      # le frontend repond (depuis le serveur)
docker compose logs -f backend    # migrations Flyway appliquees, pas d'erreur
```

Depuis un poste du reseau interne, ouvrir `http://10.0.0.10/`.

7 Premiere connexion

- Se connecter avec le compte `admin` et le mot de passe `APP_ADMIN_INITIAL_PASSWORD`.
- Changer immediatement ce mot de passe, puis creer les comptes des responsables provinciaux.
- Si la variable n'a pas ete fournie, le compte `admin` reste inutilisable par securite. Renseigner `APP_ADMIN_INITIAL_PASSWORD` puis relancer `docker compose ... up -d`.

8 Persistance et sauvegardes

Deux volumes Docker persistants :

- `db-data` : donnees PostgreSQL.
- `documents-data` : pieces jointes des sinistres (`/app/data/documents`).

Sauvegardes recommandees (quotidiennes, conservees hors du serveur) :

```
# Base de donnees
docker compose exec -T db pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" > backup-$(date
    +%F).sql

# Pieces jointes
docker run --rm -v srm-fleet_documents-data:/data -v "$PWD":/backup alpine \
  tar czf /backup/documents-$(date +%F).tar.gz -C /data .
```

Restauration : recharger le dump SQL et decompresser l'archive dans le volume correspondant.

9 Mises a jour

```
git pull
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Les nouvelles migrations Flyway s'appliquent automatiquement au demarrage du backend. Effectuer une sauvegarde (section 8) avant toute mise a jour.

10 Supervision

- Journaux : `docker compose logs -f [service]`.
- Sante : `docker compose ps` (healthchecks sur `db` et `backend`).
- Le moteur d'alertes s'execute chaque jour (planification `APP_ALERTES_CRON`, par defaut 06h00).

11 Checklist de securite (remise du projet)

- ☐ `.env` renseigne avec des secrets forts, `chmod 600`, non versionne.
- ☐ `JWT_SECRET` d'au moins 32 octets, aleatoire.
- ☐ `APP_ADMIN_INITIAL_PASSWORD` fort, mot de passe `admin` change apres la premiere connexion.
- ☐ Surcharge de production appliquee : frontend lie a l'IP interne, backend et base non publies.
- ☐ Pare-feu de l'hote limite au sous-reseau interne, aucune redirection depuis le pare-feu peripherique.
- ☐ Sauvegardes base et pieces jointes planifiees et testees (restauration verifiee).
- ☐ Acces Git au depot interne restreint a l'equipe.